ExactPDF

Comprehensive Product and Development Plan

Working title | Version 1.0 | 23 July 2026 | Product status: pre-development

Product recommendation. Build ExactPDF as a focused, privacy-first Chrome extension that converts exactly what a user selects on a webpage into a clean PDF, locally and without Chrome's print dialogue. Prioritize precision, reliability, and polished output over a broad collection of unrelated file-conversion features.

Document purpose

This document is intended to be sufficiently complete for the founder or another developer to plan, implement, test, publish, and maintain the first production release. It records the product rationale, scope boundaries, user experience, system design, standards, delivery phases, acceptance criteria, and future options. Where a detail remains unresolved, this plan specifies a default decision that can be revisited through testing rather than left ambiguous during implementation.

Contents

1. Product definition and strategy
2. Users, problems, and use cases
3. Market and competitive position
4. Scope and release boundaries
5. User experience and functional requirements
6. Technical architecture
7. Privacy, security, and compliance
8. Naming, coding, and repository conventions
9. Testing and quality assurance
10. Development roadmap
11. Chrome Web Store release plan
12. Success metrics and feedback
13. Risk register
14. Future roadmap
15. Appendices and implementation checklists

1. Product definition and strategy

1.1 Product statement

ExactPDF is a browser-content capture tool for Chrome and Chromium-based desktop browsers. It allows a user to select a webpage element, draw a capture region, capture the complete scrollable page, or extract the page's primary document content. The extension then generates a locally processed PDF with deliberate cropping, page sizing, naming, and output controls.

1.2 Core promise

Select it. Preview it. Save it exactly as a PDF - privately, directly, and without the print dialogue.

1.3 Problem definition

- Chrome's native Print > Save as PDF workflow is reliable but slow and poorly suited to capturing only one useful portion of a page.
- Printed webpages frequently include advertisements, navigation, unwanted margins, blank space, repeated headers, and poor page breaks.
- Screenshots can preserve appearance but often become awkwardly sized, low-resolution, or non-searchable documents.
- Online PDF converters can require users to upload private, authenticated, academic, or professional content to a third party.
- Existing tools often optimize either visual fidelity or clean searchable text, without making that tradeoff clear to the user.

1.4 Product principles

1. **Precision before breadth.** The extension should capture the intended content cleanly before it adds more formats or unrelated PDF utilities.
2. **Local by default.** Page content should remain on the user's device in Version 1.
3. **Explicit user control.** Access begins only after a user gesture, and the user can preview the result before downloading.
4. **Honest output modes.** The interface must explain the tradeoff between exact visual fidelity and searchable document structure.
5. **Low permission footprint.** Request only the browser permissions necessary for a visible user-facing feature.
6. **Predictable results.** Every action should produce the result implied by its label, with clear errors when a page cannot be processed.

7. **Fast first success.** A new user should complete a useful export within two minutes of installation.

1.5 Positioning

The extension should be positioned as a web-content-to-PDF capture product, not as a media downloader. The phrase 'media downloader' can imply audio or video extraction, copyright infringement, paywall circumvention, or DRM bypassing. ExactPDF serves legitimate capture, archival, study, documentation, and record-keeping workflows involving content already visible to the user.

2. Users, problems, and use cases

2.1 Initial user segments

- **Students.** Save lecture notes, learning-management-system content, diagrams, worked examples, readings, and AI conversations for annotation in Goodnotes, Notability, or Preview.
- **Researchers.** Archive articles, documentation, figures, tables, citations, and reproducible records of online sources.
- **Professionals.** Save invoices, receipts, confirmations, internal dashboards, reports, and authenticated browser views.
- **Developers and designers.** Capture interface components, code snippets, documentation sections, and visual states for reports or issue tracking.

2.2 Primary jobs to be done

- Save this exact diagram or image as a tightly cropped PDF with no white border.
- Turn this long article into a clean PDF that I can search and annotate.
- Preserve this user-visible logged-in page without uploading it to an external converter.
- Capture this entire webpage without manually scrolling and stitching screenshots.
- Save this table, post, conversation, or page section without the surrounding website.
- Create a sensibly named file that I can identify later without manually renaming it.

2.3 Representative user stories

- As a student, I can select one lecture diagram and export it at high resolution so that it imports cleanly into my notes.
- As a researcher, I can convert an article into a searchable PDF containing the source URL and capture date.

- As a professional, I can capture a receipt or confirmation from an authenticated session without sending its contents to a remote server.
- As a developer, I can select a rendered interface component and produce a visually faithful PDF for documentation.
- As a new user, I can understand the difference between Exact mode and Document mode before exporting.

3. Market and competitive position

3.1 Competitive reality

Webpage-to-PDF conversion already has established competitors. The product cannot rely on 'one-click PDF' as its sole differentiator. Its opportunity lies in combining exact selection, local processing, authenticated page support, careful cropping, strong naming defaults, and a clear choice between visual and semantic output.

Option	Current strength	Opening for ExactPDF
Chrome Print	Native, searchable text, dependable rendering	Cumbersome dialogue; weak element selection and cropping
PrintFriendly	Page cleaning and a large suite of PDF tools	Broad product; ExactPDF can be simpler and more capture-focused
PDFCrowd Save as PDF	Straightforward one-click workflow	Server processing and limited support for logged-in pages
Just-One-Page-PDF	No print dialogue; region and multi-tab options	Close competitor; differentiation must come from UX and output quality
Element PDF Extractor	Precise webpage element selection	Still relies on Chrome's print dialogue

3.2 Differentiation stack

- Exact element selection plus manual boundary adjustment
- Region, element, full-page, and clean-document capture under one narrow purpose
- Entirely local Version 1 processing
- User-visible content capture on many authenticated pages
- Automatic blank-space removal and zero-margin output
- Exact and Document export modes with an explicit tradeoff
- Source-aware filenames and optional source metadata
- A minimal, polished interface designed for a successful export in seconds

3.3 Defensibility

The capture concept itself is replicable. Sustainable advantage must therefore come from compatibility, edge-case handling, user trust, speed, output quality, recognizable branding, documentation, reviews, and continuous improvement. A large, maintained compatibility test suite can become a more meaningful advantage than keeping the source code secret.

4. Scope and release boundaries

4.1 Version 1 scope

- ☐ Element capture
- ☐ Rectangular region capture
- ☐ Complete scrollable-page capture
- ☐ Clean Document extraction
- ☐ Exact and Document PDF modes
- ☐ Preview before download
- ☐ Automatic blank-space cropping
- ☐ A4, Letter, original-size, and continuous-page output
- ☐ Portrait and landscape orientation
- ☐ Zero-margin and custom-margin controls
- ☐ Quality or compression control
- ☐ Smart filename generation
- ☐ Optional source URL and capture date metadata
- ☐ Locally stored user preferences
- ☐ Clear unsupported-page and export-failure messages

4.2 Explicit exclusions from Version 1

- Video or audio downloading
- DRM, paywall, login, or access-control circumvention
- PDF editing, signing, merging, splitting, repair, or form filling
- OCR
- Cloud synchronization or storage integrations
- User accounts
- AI summarization
- Remote page-content processing

- Unrelated image, office-document, ebook, or archive conversion

4.3 Browser and platform support

- **Required at launch.** Current stable Chrome on macOS and Windows.
- **Best-effort at launch.** Brave, Microsoft Edge, and other Chromium desktop browsers that permit Chrome Web Store extensions.
- **Not supported initially.** Chrome on iOS or Android, Safari, Firefox, restricted browser pages, and browser-internal system interfaces.

4.4 Known technical limitations to communicate

- Chrome blocks content-script injection on pages such as chrome:// URLs and portions of the Chrome Web Store.
- Cross-origin iframes may not expose their internal document structure to the extension.
- Canvas, WebGL, protected video, and highly dynamic content may be available only through visual capture.
- Infinite-scroll pages may require a user-defined stopping point or maximum capture length.
- Sticky headers, animations, lazy loading, and virtualized lists require special handling and cannot be assumed to behave like static HTML.
- Exact mode may produce a larger, image-based PDF; Document mode may simplify complex CSS.

5. User experience and functional requirements

5.1 Primary interaction model

1. The user invokes ExactPDF from the toolbar, a right-click menu, or an optional keyboard shortcut.
2. The extension receives temporary access to the active page.
3. The user chooses Select Element, Select Region, Full Page, or Clean Document.
4. ExactPDF captures or extracts the requested content.
5. A preview page opens with output mode, page sizing, margins, quality, filename, and metadata controls.
6. The user confirms the export.
7. The PDF is generated locally and downloaded.
8. A lightweight success message identifies the filename and offers another capture.

5.2 Toolbar popup

- Display the four capture methods as equally understandable actions.
- Use short labels and one-line descriptions; avoid technical language such as DOM, rasterization, or serialization.
- Remember the most recently used mode without hiding the alternatives.
- Include Settings and Help as secondary actions.
- When the active page is unsupported, disable capture actions and explain why.

5.3 Select Element mode

1. Dim the page slightly and outline the element beneath the pointer.
2. Show a small label describing the selected element in plain language where possible, such as Image, Article, Table, or Section.
3. Allow click to select, Escape to cancel, and arrow-key or compact controls to move to a parent or child element.
4. After selection, display draggable bounds when manual refinement is technically safe.
5. Capture the final selected area and open the preview.

5.4 Select Region mode

- Display a crosshair and translucent drag rectangle.
- Show live width and height in pixels.
- Permit repositioning and resizing before confirmation.
- Support Escape to cancel and Enter to confirm.
- Capture at device-pixel-ratio resolution where memory permits.

5.5 Full Page mode

- Determine the scrollable page bounds.
- Offer a brief warning for unusually long or infinite-scroll pages.
- Load lazy content progressively without racing the page.
- Temporarily suppress fixed or sticky elements during subsequent capture tiles.
- Capture overlapping tiles and stitch them without visible seams.
- Restore the user's original scroll position and temporary page changes even after failure.

5.6 Clean Document mode

- Identify the primary article or document content.

- Preserve headings, paragraphs, lists, links, images, code blocks, tables, and captions when practicable.
- Remove navigation, advertisements, cookie banners, unrelated sidebars, and page chrome.
- Present a readable preview with page-break controls.
- Fall back to Exact mode when no sufficiently confident document structure is found.

5.7 Export modes

Mode	Optimizes for	Expected tradeoff	Best uses
Exact	Visual fidelity	Mostly image-based; larger file; text may not be selectable	Diagrams, interfaces, unusual layouts, charts
Document	Searchable text and pagination	May simplify complex styling or interactive content	Articles, readings, documentation, conversations

5.8 Preview and export controls

- Mode: Exact or Document
- Page size: Original, Continuous, A4, Letter
- Orientation: Automatic, Portrait, Landscape
- Margins: Zero, Narrow, Standard, Custom
- Background: Preserve, Force light, Transparent where supported
- Quality: Compact, Balanced, Maximum
- Filename: Editable suggested filename
- Metadata: Source URL, page title, capture date, and optional footer
- Page breaks: Automatic with manual break controls in Document mode

5.9 Filename convention

Default pattern: {sanitized-page-title}_{content-label}_{YYYY-MM-DD}.pdf. The extension should remove filesystem-invalid characters, collapse repeated whitespace, trim trailing punctuation, and cap the base filename at a reasonable length. If the title is unavailable, use the website hostname. If a duplicate exists, allow Chrome's normal duplicate-number behavior unless the user has enabled a timestamp suffix.

- Example: MAT137_Lecture_Integration_Example_2026-07-23.pdf
- Example: developer.chrome.com_activeTab_2026-07-23.pdf
- Example: ChatGPT_DriveSync_Planning_2026-07-23.pdf

5.10 Error handling

- **Unsupported page.** Explain that Chrome does not allow extensions to access the current browser page.
- **Capture too large.** Offer a lower-quality export, a selected region, or a defined stopping point.
- **Document structure not found.** Offer Exact Full Page mode immediately.
- **Cross-origin content unavailable.** Offer visual region capture when permissible.
- **Memory or rendering failure.** Restore the page state, discard temporary data, and provide a retry with safer settings.
- **Partial lazy loading.** Tell the user which content may be missing and allow a recapture after scrolling.

6. Technical architecture

6.1 Recommended stack

- Manifest V3
- TypeScript with strict compiler settings
- Vite for bundling
- Preact or lightweight React for popup, preview, and settings interfaces
- Chrome extension APIs for active-tab access, script injection, preferences, menus, and download initiation
- A local PDF library such as pdf-lib or pdfmake after a focused rendering proof of concept
- Mozilla Readability or an equivalent audited extraction component for Document mode
- DOM sanitization before preview reconstruction
- Vitest for units and Playwright for end-to-end browser tests

6.2 Component model

- **Popup.** Starts a capture, displays availability, and opens settings or help.
- **Service worker.** Coordinates browser events, messages, temporary state, and capture sessions.
- **Selection content script.** Provides element and region overlays and collects capture geometry.
- **Capture engine.** Produces visual tiles or a sanitized document representation.
- **Preview application.** Displays the result and collects export settings.
- **PDF renderer.** Converts the visual or document representation into PDF bytes locally.

- **Settings store.** Persists non-sensitive preferences and schema version.
- **Diagnostics layer.** Creates local, privacy-safe error codes without recording page contents.

6.3 Data flow

1. A user gesture activates temporary access to the current tab.
2. The service worker creates a capture-session identifier and injects only the script required for the chosen mode.
3. The content script returns capture geometry, sanitized document data, or tile instructions.
4. Temporary capture data is transferred to the preview application.
5. The local rendering worker generates the PDF.
6. The browser downloads the resulting Blob.
7. The session data is deleted from memory and temporary storage.

6.4 Exact capture engine

- Use visual capture for region and complex-layout fidelity.
- Respect device pixel ratio while enforcing memory and dimension limits.
- For long pages, capture tiles with a small overlap and deterministic seam removal.
- Freeze or pause animations where practical.
- Hide the extension's own overlays before capture.
- Suppress repeated fixed elements after the first tile.
- Restore scroll position, animations, and modified styles in a finally-style cleanup path.
- Generate image-based PDF pages or a continuous custom-size page according to the selected preset.

6.5 Document capture engine

- Clone rather than mutate the live page.
- Sanitize scripts, event handlers, forms, embeds, and unsafe URLs.
- Resolve relative links and image sources against the page URL.
- Extract semantic structure and meaningful alt text.
- Convert supported structures into explicit PDF primitives so text remains selectable.
- Use consistent typography and page-break rules instead of attempting to reproduce every CSS declaration.
- Provide a confidence score internally and recommend Exact mode below the chosen threshold.

6.6 Permissions

- **activeTab.** Temporary access after an explicit user gesture.
- **scripting.** Inject the selection or extraction code into the active tab.
- **storage.** Save settings, presets, and schema version.
- **contextMenus.** Optional; enables right-click actions such as Save image or Save selected element.

Avoid <all_urls>, debugger, webRequest, persistent tabs access, browsing history, cookies, and broad host permissions in Version 1. If later testing proves an additional permission necessary, document the exact feature requiring it, request it at runtime where possible, update the privacy disclosure, and test the installation warning before release.

6.7 Storage model

- Persist only preferences, preset definitions, onboarding state, and a settings schema version.
- Keep capture bytes and extracted page content in memory or short-lived extension storage.
- Delete temporary content after download, cancellation, timeout, extension reload, or handled failure.
- Do not retain URLs, titles, filenames, or export history unless a future visible feature explicitly requires it.

6.8 Performance budgets

- Popup interactive within 200 ms on a typical modern desktop after warm load.
- Selection overlay responsive at approximately display refresh rate without forcing continuous full-page layout calculations.
- Initial region preview within two seconds for ordinary viewport-sized content.
- No long-running work on the UI thread when a worker or offscreen context is suitable.
- Configurable maximum full-page pixel area with an intelligible fallback rather than a browser crash.
- Extension bundle kept reviewable and free of unused heavyweight dependencies.

7. Privacy, security, and compliance

7.1 Privacy commitment

Version 1 should transmit no captured page content, screenshots, extracted text, URLs, titles, or PDF bytes to the developer or a third party. All conversion should occur locally. The privacy

policy and Store listing must state this accurately without claiming broader security guarantees than the implementation provides.

7.2 Required safeguards

- ☐ Use activeTab instead of persistent access wherever possible.
- ☐ Bundle all executable code with the extension.
- ☐ Sanitize cloned page content before displaying it in an extension preview.
- ☐ Block javascript:, data-executable, and unsafe embedded content in reconstructed documents.
- ☐ Never evaluate page-provided strings as code.
- ☐ Use a restrictive extension content security policy.
- ☐ Audit third-party dependencies and pin versions through a lockfile.
- ☐ Add automated dependency and static-analysis checks.
- ☐ Remove captured content from temporary storage after each session.
- ☐ Document every requested permission in the README and Store privacy form.

7.3 Copyright and access boundaries

- Do not market the extension as a means of bypassing paywalls, login restrictions, DRM, or download controls.
- Do not add extraction logic specifically designed for protected streaming audio or video.
- Do not imply that installing the extension grants rights to reproduce or redistribute captured content.
- Include a concise acceptable-use statement that users are responsible for respecting copyright, confidentiality, and website terms.
- Respond promptly to legitimate intellectual-property complaints involving the product's branding, listing, or functionality.

7.4 Analytics decision

Launch without page-level analytics. Use Chrome Web Store aggregate installation and engagement information, support messages, opt-in beta feedback, and privacy-safe local error codes. If product analytics are later introduced, they must be opt-in or appropriately disclosed, must never include captured content or full URLs, and must directly support a visible product purpose.

8. Naming, coding, and repository conventions

8.1 Product naming

- **Working name.** ExactPDF.
- **Naming goal.** Communicate precision and PDF output without implying affiliation with Chrome, Google, Adobe, or another company.
- **Before finalization.** Check Chrome Web Store conflicts, web search results, domain availability, social handles, and relevant trademarks.
- **Descriptor.** Select and save web content as PDF.

8.2 User-facing terminology

Use	Avoid	Reason
Web content	Browser media	Clearer and less associated with protected audio or video
Exact mode	Raster mode	Describes the benefit rather than the implementation
Document mode	DOM extraction	Understandable to non-technical users
Select Element	Node picker	Matches the user's task
Full Page	Scroll stitching	Describes the result
Processed locally	Zero-risk or completely secure	Accurate and supportable privacy language

8.3 TypeScript conventions

- Use strict TypeScript and prohibit implicit any.
- Use PascalCase for types and UI components, camelCase for functions and variables, and SCREAMING_SNAKE_CASE only for true constants.
- Name asynchronous commands with action verbs: captureFullPage, buildDocumentPdf, restorePageState.
- Use discriminated unions for capture modes and result states.
- Define serializable message contracts in one shared module.
- Represent expected failures as typed results rather than uncaught exceptions.
- Keep Chrome API calls behind small adapters so browser behavior can be mocked in tests.
- Add comments for non-obvious browser constraints, not for code that already explains itself.

8.4 Suggested domain types

- CaptureMode = 'element' | 'region' | 'fullPage' | 'document'
- RenderMode = 'exact' | 'document'
- PageSize = 'original' | 'continuous' | 'a4' | 'letter'
- CaptureSession
- CaptureBounds
- VisualCapturePayload
- DocumentCapturePayload
- ExportOptions
- ExportResult
- ExactPdfErrorCode

8.5 Repository layout

Recommended structure:

```
src/background/          service worker and orchestration
src/content/             selection overlays and page extraction
src/capture/exact/       region and tiled visual capture
src/capture/document/    semantic extraction and sanitization
src/pdf/                 PDF rendering and page-layout logic
src/preview/             preview application
src/popup/               toolbar popup
src/options/             settings interface
src/shared/              contracts, errors, utilities, constants
tests/unit/              deterministic module tests
tests/integration/       component and Chrome-adapter tests
tests/e2e/               browser workflows and fixture pages
tests/fixtures/sites/    controlled difficult-page fixtures
docs/                    architecture, privacy, compatibility, decisions
store-assets/            icon and listing source assets
```

8.6 Git and documentation conventions

- Keep the repository public at launch or shortly afterward unless a specific commercial reason arises.
- Do not store credentials, Store tokens, private keys, analytics secrets, or payment secrets in the repository.
- Use focused commits and descriptive imperative commit subjects.
- Do not allow automated coding tools to create commits without review.
- Maintain README, CONTRIBUTING, SECURITY, PRIVACY, CHANGELOG, and compatibility documentation.
- Record consequential architecture decisions as short decision records under docs/decisions.
- Use issues for identifiable defects and milestones rather than burying unfinished work in comments.

9. Testing and quality assurance

9.1 Quality bar

The extension is ready for public release only when ordinary exports are fast and predictable, failures are recoverable, temporary page modifications are always restored, and the output matches the preview. A feature that works on a single demonstration page is not production-ready.

9.2 Test layers

- **Unit tests.** Filename sanitization, dimensions, page sizing, tile overlap, crop calculations, settings migrations, error mapping, and document transformations.
- **Integration tests.** Capture-session messaging, service-worker restarts, preview handoff, storage adapters, and PDF renderer inputs.
- **End-to-end tests.** Real extension installation, user invocation, element selection, region selection, full-page capture, preview changes, and downloaded PDF verification.
- **Visual regression tests.** Pixel or structural comparison of deterministic fixture-page exports.
- **Manual compatibility tests.** Representative public and authenticated workflows across Chrome, Brave, and Edge.

9.3 Website compatibility matrix

Page category	Required scenario	Principal failure risks
Static article	Long article with headings, links, and images	Page breaks; missing fonts or images
Documentation	Code blocks, side navigation, anchors	Overflow; syntax colors; unwanted navigation
AI conversation	Long mixed text, code, tables, citations	Virtualization; lazy loading; wide blocks
LMS or authenticated page	Visible notes or assignment content	Frames; permissions; private content handling
Data table	Wide table with sticky header	Cropping; pagination; repeated header
Image-heavy page	Responsive images and lazy loading	CORS; resolution; incomplete loading
Infinite scroll	Long feed with user-defined endpoint	Unbounded capture; duplicated content
Canvas or chart	Rendered chart without accessible DOM	Visual-only fallback; resolution
Dark mode	Dark article and interface capture	Background handling; unreadable forced-light result
Complex fixed layout	Sticky headers and floating controls	Repeated overlays; seams

9.4 Core acceptance tests

☐ A first-time user can export a selected element within two minutes.

☐ A selected element is cropped to the expected bounds within the documented tolerance.

☐ A region capture matches the on-screen region at high-DPI resolution.

☐ A long static page exports without duplicated tiles or visible seams.
☐ Fixed headers are not repeated throughout a full-page export.
☐ Document mode produces selectable text, working links, and readable headings.
☐ Cancel and failure paths restore the original page scroll position and styles.
☐ No captured content is transmitted over the network by the extension.
☐ The extension functions without <all_urls> or debugger permissions.
☐ Generated filenames contain no invalid filesystem characters.
☐ The preview corresponds to the downloaded file's orientation, margins, and page size.
☐ The extension remains usable after the Manifest V3 service worker is suspended and restarted.

9.5 PDF validation

- Open every golden export in Chrome PDF Viewer and macOS Preview.
- Confirm page count, dimensions, orientation, metadata, image clarity, and absence of blank trailing pages.
- For Document mode, verify text selection, search, hyperlinks, line wrapping, and page breaks.
- For Exact mode, inspect seams, scaling, cropping, repeated fixed elements, and image compression.
- Test import into Goodnotes or another annotation workflow before launch.

10. Development roadmap

10.1 Delivery strategy

Use vertical milestones that each produce a demonstrable user outcome. Do not build all infrastructure before testing the central capture risk. The first technical proof should compare candidate PDF-generation methods on difficult content and settle the Exact versus Document architecture.

Phase	Objective	Exit criterion
0. Validation	Reproduce competitor workflows and test rendering approaches	Architecture decision and 10-page fixture set
1. Foundation	Manifest, popup, session model, element selection	Selected element exports locally
2. Capture engine	Region capture, cropping, image quality	High-DPI region and element PDFs

Phase	Objective	Exit criterion
3. Full page	Tile capture, lazy content, sticky suppression	Long fixture pages export without seams
4. Document mode	Semantic extraction and searchable rendering	Representative articles export with selectable text
5. Product polish	Preview, presets, errors, naming, settings	Complete V1 user workflow
6. Release	Compatibility, privacy, assets, beta, submission	Approved Store listing and public repository

10.2 Phase 0 - technical validation

- ☐ Install and test the leading competing extensions on the same fixture pages.
- ☐ Document which outputs are searchable, image-based, server-processed, or sent through print preview.
- ☐ Prototype at least two visual capture approaches and two document-rendering approaches.
- ☐ Measure output clarity, speed, memory, bundle size, and Chrome permission requirements.
- ☐ Choose PDF libraries only after rendering representative CSS, fonts, images, code blocks, and long pages.
- ☐ Write the first architecture decision record.

10.3 Phase 1 - foundation

- ☐ Create Manifest V3 extension shell.
- ☐ Configure TypeScript, formatting, linting, tests, and production builds.
- ☐ Implement toolbar popup and temporary active-tab access.
- ☐ Create capture-session message contracts.
- ☐ Build element hover, parent selection, confirmation, and cancellation.
- ☐ Generate and download one locally produced element PDF.

10.4 Phase 2 - element and region capture

- ☐ Implement adjustable region selection.
- ☐ Handle device pixel ratio and viewport scaling.
- ☐ Hide extension overlays during capture.
- ☐ Add automatic crop and blank-space detection.
- ☐ Implement original-size and standard-paper PDF outputs.
- ☐ Add visual regression fixtures for images, charts, and interface components.

10.5 Phase 3 - full-page capture

- ☐ Build scroll-tile capture and seam alignment.
- ☐ Restore scroll position reliably.
- ☐ Suppress duplicate sticky elements.
- ☐ Handle lazy-loaded images with bounded waits.
- ☐ Implement maximum-size safeguards and long-page warnings.
- ☐ Test static, sticky, virtualized, and infinite-scroll fixtures.

10.6 Phase 4 - Document mode

- ☐ Extract main content into a sanitized intermediate representation.
- ☐ Render headings, text, lists, links, images, tables, and code blocks.
- ☐ Implement page breaking and font fallback.
- ☐ Add document-extraction confidence and Exact-mode fallback.
- ☐ Verify selectable text and hyperlinks.
- ☐ Test articles, documentation, AI conversations, and LMS content.

10.7 Phase 5 - product polish

- ☐ Build preview application and export controls.
- ☐ Implement filename generation and editable naming.
- ☐ Add page metadata options.
- ☐ Persist settings with a versioned schema.
- ☐ Write onboarding, help, limitations, and error messages.
- ☐ Complete accessibility keyboard flows and focus management.

10.8 Phase 6 - release

- ☐ Complete the compatibility matrix on stable Chrome and Brave, then Edge where available.
- ☐ Run security and dependency review.
- ☐ Finalize privacy policy, acceptable-use language, and support process.
- ☐ Prepare icon, screenshots, promotional images, and demonstration video.
- ☐ Run a 10-20 person beta and resolve release-blocking defects.
- ☐ Package a clean production ZIP and verify it through Load unpacked before submission.
- ☐ Submit to the Chrome Web Store and prepare the public GitHub release.

11. Chrome Web Store release plan

11.1 Proposed listing

Name: ExactPDF - Select and Save Web Content as PDF

Short description: Select any webpage element, region, image, or complete page and save it as a clean PDF - locally and without the print dialogue.

Single purpose: ExactPDF captures user-selected web content and converts it into a locally generated PDF.

11.2 Listing assets

- ☐ Simple recognizable icon at every required size
- ☐ Screenshot of the four capture actions
- ☐ Screenshot of element selection
- ☐ Screenshot of the preview and Exact/Document choice
- ☐ Before-and-after example showing unwanted page chrome removed
- ☐ Short demonstration video
- ☐ Public privacy policy URL
- ☐ Support email and support or issue-report page

11.3 Submission package

- ☐ Production Manifest V3 build
- ☐ No source maps containing sensitive or unnecessary source material
- ☐ No development endpoints or localhost permissions
- ☐ No remotely loaded executable code
- ☐ Accurate permission justifications
- ☐ Accurate data-use disclosures
- ☐ Reviewer instructions for each capture mode
- ☐ Test page or deterministic reviewer workflow
- ☐ Version number and changelog
- ☐ Two-step verification enabled on the publisher account

11.4 Review expectations

New extensions and developers can receive closer review, particularly when an extension requests broad or sensitive permissions. Keep the code reviewable, permissions narrow, disclosures consistent, and reviewer instructions explicit. Allow several weeks of schedule margin even though many reviews complete sooner. Do not announce a fixed public launch date until approval is received.

11.5 Public repository plan

- Publish at launch or shortly afterward.
- Include an animated demonstration above the README fold.
- Explain local processing and permissions plainly.
- Publish the architecture, compatibility table, roadmap, and build instructions.
- Use a recognized open-source licence selected deliberately before accepting contributions.
- Keep any future payment or licensing service in a separate repository.
- Treat the public issue tracker as part of customer support and product research.

12. Success metrics and feedback

12.1 Product quality metrics

- First-export completion rate
- Median time from installation to first successful PDF
- Export success rate by capture mode
- Failure code frequency
- Average PDF generation time by output size
- Store rating and review themes
- Weekly active users relative to total installs
- Four-week retention

12.2 Initial targets

- More than 90% successful exports across the maintained compatibility suite
- Median first successful export under two minutes
- Ordinary export failure rate below 5%
- Chrome Web Store rating of 4.5 or higher after a meaningful review sample
- Weekly-active-to-installed-user ratio above 30%

- Four-week retention above 20%

12.3 Feedback loop

1. Provide a visible Report a problem action that includes only a local error code by default.
2. Ask users to describe the page category and result without automatically attaching page contents.
3. Convert repeated failures into controlled fixture pages.
4. Add regression tests before closing compatibility defects.
5. Publish significant fixes in a concise changelog.
6. Prioritize improvements by frequency, severity, and fit with the single product purpose.

13. Risk register

Risk	Impact	Mitigation
Existing competitors cover basic conversion	Weak differentiation and limited adoption	Lead with exact local selection, dual modes, naming, and superior reliability
Arbitrary webpages are difficult to render	Inconsistent output	Maintain two export modes, a compatibility suite, and explicit fallbacks
Long pages exhaust memory	Browser slowdown or crash	Pixel budgets, tiling, workers, warnings, and lower-quality retry
Broad permissions reduce trust	Lower installs and slower review	Use activeTab and runtime access; avoid debugger and all-URLs access
Private page content leaks	Severe trust and compliance failure	No backend, strict sanitization, short-lived data, network tests
Store rejection	Delayed launch	Single-purpose scope, accurate disclosures, reviewer steps, minimal code and permissions
Copyright misuse perception	Takedown or reputational damage	Avoid downloader branding and protected-media or bypass features
Open-source clones	Copying and confusion	Build brand, support, reviews, compatibility, and verified publisher identity
Service-worker suspension breaks flows	Intermittent failures	Persist minimal session checkpoints and test worker restarts
Document mode loses complex styles	User disappointment	Explain mode tradeoff and provide immediate Exact-mode fallback

14. Future roadmap

14.1 Near-term enhancements after product validation

- Combine several selected captures into one PDF
- Multi-tab batch export
- Custom export presets
- Manual page-break editor
- Improved sticky-element and cookie-banner detection
- Additional filename templates
- Optional right-click image-to-PDF workflow
- Direct handoff to common note-taking or storage workflows where browser APIs permit it

14.2 Later capabilities requiring separate evaluation

- OCR for image-based captures
- PDF compression and optimization
- Searchable text overlay for Exact captures
- Team-managed presets
- Cloud synchronization
- Firefox or Safari versions
- PNG or JPEG export as capture-related companion formats
- Web-archive export such as MHTML, if it remains consistent with the product's single capture purpose

14.3 Feature admission rule

A proposed feature should be added only if it directly improves selecting, preserving, organizing, or exporting web content. If it primarily edits an existing PDF, downloads protected media, or converts unrelated file types, it belongs in a separate product.

14.4 Monetization note

Monetization is not a launch priority. Version 1 should be free and should optimize for product quality, user trust, adoption, reviews, and portfolio value. Paid features should be considered only after usage demonstrates which advanced workflows users repeatedly request. Advertising and sale of browsing data are inconsistent with the product's positioning and should be excluded.

15. Appendices and implementation checklists

Appendix A - Definition of done for Version 1

- ☐ All four capture modes are implemented or an explicitly deferred mode has been removed from the public promise.
- ☐ Exact and Document modes behave as described.
- ☐ Preview controls change the downloaded output correctly.
- ☐ The maintained compatibility suite meets the release success threshold.
- ☐ No page content leaves the device during capture or export.
- ☐ All temporary page changes are restored after success, cancellation, and failure.
- ☐ Permissions are limited to the documented set.
- ☐ Privacy policy, Store disclosures, README, and implementation agree.
- ☐ The extension has completed a representative external beta.
- ☐ The production package has been loaded locally and tested from a clean browser profile.
- ☐ Store assets and reviewer instructions are complete.
- ☐ The public repository contains no secrets and has reproducible build instructions.

Appendix B - Pre-submission checklist

- ☐ Increment and verify the extension version.
- ☐ Run formatting, linting, type checking, unit tests, integration tests, and end-to-end tests.
- ☐ Run the full compatibility matrix.
- ☐ Build from a clean checkout.
- ☐ Inspect the production ZIP contents.
- ☐ Test the ZIP through Chrome's Load unpacked flow.
- ☐ Review every permission and corresponding user warning.
- ☐ Review network activity while capturing private fixture content.
- ☐ Confirm no remote executable code or development URLs.
- ☐ Proofread Store title, description, privacy answers, and screenshots.
- ☐ Verify support contact information.
- ☐ Archive the submitted ZIP and commit hash.

Appendix C - First development session

1. Create the project directory and initialize version control.
2. Set up Manifest V3, TypeScript, Vite, linting, formatting, and tests.

3. Create a minimal popup with one Capture Element action.
4. Implement activeTab and scripting permissions only.
5. Inject an element-hover overlay into a controlled fixture page.
6. Return the selected bounds to the service worker.
7. Capture the selected region and generate a local proof-of-concept PDF.
8. Open the result in Chrome PDF Viewer and Preview.
9. Record fidelity, text-searchability, speed, and memory findings.
10. Use the findings to choose the first rendering library and write the architecture decision.

Appendix D - Key decisions requiring prototype evidence

- Which library produces the best balance of local rendering, searchable text, bundle size, and Manifest V3 compatibility?
- Should full-page Exact exports use one continuous PDF page by default or paginate automatically?
- How accurately can fixed and sticky elements be detected without breaking legitimate positioned content?
- What maximum pixel area is safe across ordinary Windows and macOS hardware?
- How should virtualized AI conversations and feeds be expanded without causing unbounded capture?
- What Document-mode structures can be supported reliably in V1?
- Can downloads be initiated cleanly without requesting the broader downloads permission?

Appendix E - Sources and policy references

- Chrome Web Store Program Policies
- Chrome extension single-purpose guidance
- Chrome activeTab documentation
- Manifest V3 overview
- Chrome Web Store review process
- Chrome Web Store publishing documentation
- Chrome developer account registration
- PrintFriendly Chrome Web Store listing
- Just-One-Page-PDF Chrome Web Store listing
- Element PDF Extractor Chrome Web Store listing
- PDFCrowd Save as PDF Chrome Web Store listing

Appendix F - Maintenance cadence

- **Every release.** Run the regression suite, review permissions, update the changelog, and verify the production package.
- **Monthly during active growth.** Review Store feedback, compatibility failures, dependency alerts, and Chrome platform changes.
- **Quarterly.** Reassess the roadmap, audit privacy disclosures, test major Chromium versions, and retire unreliable dependencies.
- **After a policy or permission change.** Review the full Store listing and privacy policy before submitting the update.